PRACTICAL LECTURE

Building Your First AI Agent

Using LangChain + Groq (100% Free)

Hands-on Session • VS Code • No Credit Card Needed

AGENDA

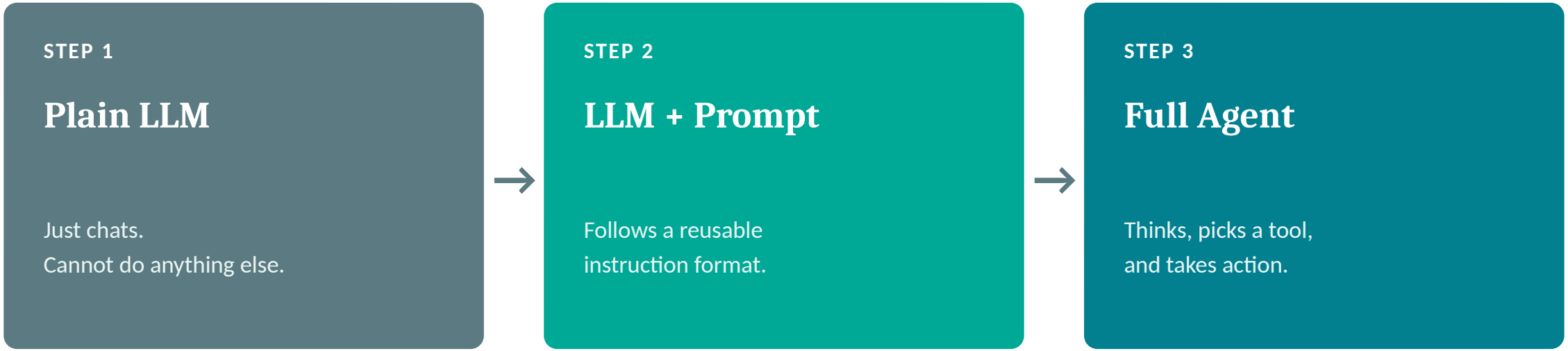
What We'll Do Today

1. Setup: Groq API Key + VS Code
2. Step 1 — Talk to an LLM
3. Step 2 — Add a Prompt Template
4. Step 3 — Turn it into an Agent
5. Your Turn: Build Your Own Tool
6. Common Errors & Fixes

GOAL

What Are We Building?

By the end of this class, you will have built a working AI Agent that can use a tool to answer questions the LLM cannot answer on its own.



The final test: We ask "What is the weather in Lahore?" — Step 1 fails, Step 3 answers correctly. That difference IS the agent.

SETUP

Why Are We Using Groq?



Groq is a company that runs open-source AI models (like Llama) on special hardware. It gives us a free API to use them.

Free to start

No credit card required — perfect for learning and classroom use.

Very fast

Replies come back almost instantly, so you are not waiting during class.

Good models

Runs strong open models such as Llama, ready to use out of the box.

Same LangChain code

If you switch to OpenAI later, only one line of code changes.

Note: Groq (with a q) is different from Grok (with a k) by xAI. They are separate companies.

SETUP

Setup 1 — Get Your Free Groq API Key

1	**Go to the website**	Open console.groq.com in your browser.
2	**Sign up**	Create a free account using Google or email. No payment details needed.
3	**Open API Keys**	From the left side menu, click on the "API Keys" section.
4	**Create key**	Click "Create API Key", give it any name, and click create.
5	**Copy and save it**	Copy the key immediately and paste it in a notepad. You cannot see it again later.

Warning: Never share your API key or post it publicly. Treat it like a password.

SETUP

Setup 2 — Project Folder, venv & Install

In VS Code, open a new folder, then open its built-in Terminal (Ctrl + `) and run these one by one:

```
python -m venv venv
venv\Scripts\activate      # Windows
source venv/bin/activate  # Mac / Linux

pip install langchain langchain-groq python-dotenv
```

venv	Creates an isolated Python environment just for this project, so packages don't clash with other projects.
activate	Turns the venv on. VS Code should show "(venv)" in the terminal once it works — always check this.
langchain / langchain-groq	The framework, and the connector that lets it talk to Groq's models.
python-dotenv	Lets us safely load our API key from a separate file — explained on the next slide.

Tip: also install the "Python" extension (by Microsoft) in VS Code — it gives you Run buttons and IntelliSense.

SETUP

Setup 3 — Store Your Key Safely (.env file)

In your project folder, create a new file named exactly .env and add this one line to it:

```
GROQ_API_KEY=paste_your_key_here
```

Then, at the top of your Python file (e.g. app.py), add these two lines before anything else:

```
from dotenv import load_dotenv
load_dotenv()   # reads .env and loads GROQ_API_KEY automatically
```

Why do we do it this way?

We never write the key directly in our .py file. If you push that file to GitHub, anyone can steal your key. The .env file stays only on your computer, and load_dotenv() quietly loads it into os.environ, where LangChain automatically finds GROQ_API_KEY on its own.

Important: never commit the .env file to Git. Add a file named .gitignore with the line ".env" inside it.

HANDS-ON

Step 1 — Just Talk to the LLM

Create a file named step1.py in VS Code and write:

```
from dotenv import load_dotenv
load_dotenv()
from langchain.chat_models import init_chat_model

model = init_chat_model("groq:llama-3.3-70b-versatile")

reply = model.invoke("Lahore ke baare mein 2 lines likho")
print(reply.content)
```

Expected output:

```
Lahore Pakistan ka dusra sabse bara shehar hai aur Punjab ka darul hukumat hai.
```

Run it from the VS Code terminal: `python step1.py`

EXPLANATION

Step 1 — Line by Line

`init_chat_model`

A helper that sets up a chat model for us. It works with Groq, OpenAI, Anthropic — all the same way.

`"groq:llama-3.3-70b-versatile"`

Two parts: "groq" is the provider, and after the colon is the model name. Llama 3.3 70B is a strong free model.

`.invoke(...)`

Means "run it once with this input". We send our question and wait for the reply.

`reply.content`

The reply is an object with extra info inside. .content pulls out just the text we want to read.

TRY THIS IN CLASS

Now Find the Limitation

Ask the same model something it cannot possibly know. Run this:

```
print(model.invoke("Lahore ka abhi mausam kya hai?").content)
```



What happens

The model says it cannot check live weather, or it makes up a guess. Both are wrong answers.



Why it happens

An LLM only knows what it was trained on. It has no way to reach the outside world. To fix this, we must give it a TOOL.

Keep this file open — we will come back and beat it in Step 3.

HANDS-ON

Step 2 — Add a Prompt Template

Add this below your existing code in the same file (model is already defined above):

```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_template(
    "Tum ek teacher ho. {topic} ko student ko asaan alfaz mein samjhao."
)

chain = prompt | model

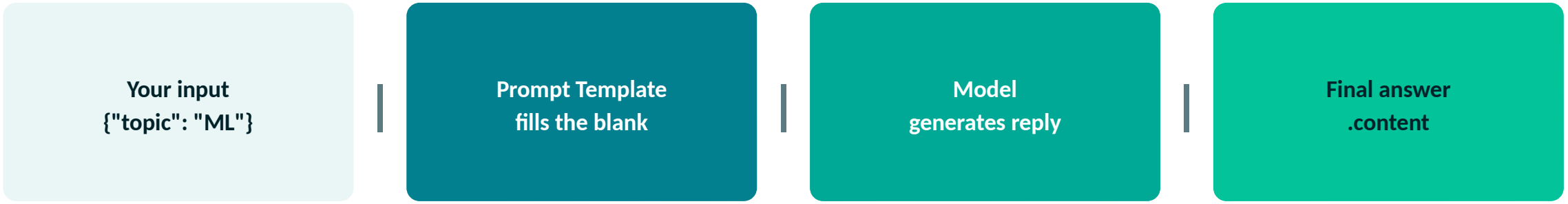
print(chain.invoke({"topic": "Machine Learning"}).content)
```

Now change only the topic and run again: **"Photosynthesis", "Blockchain", "Database".** Same template, different input — that is the whole point of a template.

EXPLANATION

Step 2 — What Is a Chain?

The pipe symbol | connects pieces together. Output of the left side becomes input of the right side.



{topic}

A blank space in the text. Whatever you pass in gets placed here before sending to the model.

Why use templates?

So you write the instruction once and reuse it a hundred times with different inputs. No copy-paste.

This is a "chain"

Two steps joined in order. In the theory lecture, this was the LangChain "Chains" building block.

HANDS-ON

Step 3 — Turn It Into an Agent

Same file, add this below (model is already defined above):

```
from langchain.agents import create_agent

def get_weather(city: str) -> str:
    """Kisi bhi shehar ka mausam batata hai."""
    return f"{city} mein abhi 32 degree hai aur dhoop hai."

agent = create_agent(
    model="groq:llama-3.3-70b-versatile",
    tools=[get_weather],
    system_prompt="Tum ek madadgar assistant ho."
)
```

Note: we return fake weather data on purpose. Today the goal is to see the tool get called — not to build a real weather service.

HANDS-ON

Step 3 — Now Run It

```
result = agent.invoke({
    "messages": [{"role": "user", "content": "Lahore ka mausam?"}]
})

print(result["messages"][-1].content)
```

Expected output:

```
Lahore mein abhi 32 degree hai aur dhoop hai.
```

This is the moment. Same question that failed in Step 1 now works — because the model was given a tool and it decided, on its own, to use it.

EXPLANATION

Step 3 — What Just Happened Inside?

1	**You ask**	"Lahore ka mausam?" goes to the agent.
2	**Model thinks**	"I do not know live weather, but a get_weather tool is available."
3	**Tool runs**	Python actually executes get_weather("Lahore") and returns the text.
4	**Model replies**	It reads the tool result and writes a natural sentence back to you.

That 4-step loop is exactly the Think → Act → Observe → Reply cycle from the theory lecture.

IMPORTANT

One Small Thing That Matters a Lot

How does the model know when to use your tool? It reads the function name and the docstring.

Bad — no docstring

```
def gw(c):
    return "32 degree"
```

The model has no idea what this does, so it will probably never call it.

Good — clear name + docstring

```
def get_weather(city: str) -> str:
    """Shehar ka mausam batata hai."""
    return "32 degree"
```

Now the model clearly knows what the tool is for and when to use it.

Rule for students: always give your tool a meaningful name, add type hints (city: str), and write one clear line in the docstring saying what it does. The docstring is not a comment — it is the instruction the AI reads.

EXERCISE

Your Turn — Build Your Own Tool

Write a tool of your own and plug it into the agent. Try one of these:

Calculator

A tool that takes two numbers and returns their multiplication.

Currency

A tool that converts dollars to rupees using a fixed rate.

Student marks

A tool that takes a name and returns that student's marks from a dictionary.

```
def multiply(a: int, b: int) -> int:
    """Do numbers ko multiply karta hai."""
    return a * b

agent = create_agent(model="groq:llama-3.3-70b-versatile",
                     tools=[get_weather, multiply])
```

Notice: tools is a list — you can give the agent as many tools as you want.

TROUBLESHOOTING

Common Errors and How to Fix Them

`ModuleNotFoundError`	Your venv is not activated, or you forgot to run pip install. Check the terminal shows "(venv)".
`AuthenticationError / 401`	Your API key is wrong, or the .env file name/spelling is off. It must be named exactly ".env".
`RateLimitError / 429`	Free tier limit hit. Wait a minute, or ask students to space out their runs.
`Model not found`	That model name is retired. Check console.groq.com for the current model list and update the string.
`Agent ignores the tool`	Your docstring is missing or unclear. Rewrite it so it plainly says what the tool does.

WRAP UP

Recap of Today

1. Groq gives us a free, fast LLM to practice with
2. init_chat_model connects LangChain to any provider in one line
3. model.invoke() sends one question and returns one answer
4. A prompt template lets you reuse one instruction with many inputs
5. The pipe symbol | joins steps together into a chain
6. create_agent + a tool turns a plain LLM into a real agent
7. A clear docstring is what tells the model when to use your tool



You Built an Agent!

Next class: memory, real APIs, and moving to LangGraph.